A FORMALLY VERIFIED HEXADECIMAL DIGIT EXTRACTION FOR π WITH AN EXPLICIT GUARD FOR EVERY POSITION

BRIAN BERNARD TAYLOR

Abstract. We describe a Lean 4 development, built on Mathlib, about the Bailey–Borwein–Plouffe (BBP) formula for π. It contains an independent proof that the BBP series equals π, an integer-only certificate for the hexadecimal digit $d_n = \lfloor 16^n \pi \rfloor \bmod 16$, and a soundness theorem: when the two ends of the certificate's bracket agree, the certified digit is the true digit. To remove the dependence on that check, we formalize an explicit irrationality-measure bound at the denominators 16^k: for every $k \geq 1$ and every integer p, $|16^k\pi - p| \cdot 16^{570k} > 2^{-7182802} \cdot 16^k$. Our main theorem shows that for every $n \geq 0$ the certificate returns d_n at every guard depth $g \geq 569n + \lfloor \log_{16}(n+1) \rfloor + 1 + B$ with $B = 1\,795\,710$; a simplified linear form of the guard is also proved. We also prove a short chain of exact statements relating the run of 0s or Fs after position n, the distance from $16^n\pi$ to the nearest integer, and the least sufficient guard depth. All statements are for every n or for explicit finite ranges; no asymptotic, probabilistic or normality claim is made. The proofs use only the axioms `propext`, `Classical.choice` and `Quot.sound`, and no `native_decide`.

1. Introduction

The BBP formula [1]

$$\pi = \sum_{k \geq 0} \frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right) \tag{1}$$

lets one compute the hexadecimal digit of π at position n without computing the earlier digits. Every implementation computes a finite sum to finite precision, so the algorithm needs a *guard*: enough extra digits that the rounding error cannot change the digit being extracted. How many extra digits are enough depends on how close $16^n\pi$ is to an integer, which is a Diophantine question about π.

This paper reports a Lean 4 [4] development, built on Mathlib [10], that makes the guard explicit and proves it sufficient for every position. We use the following conventions throughout. For $n \in \mathbb{N}$ let

$$x_n = 16^n\pi, \qquad d_n = \lfloor x_n \rfloor \bmod 16, \qquad \delta_n = \min\big(\{x_n\},\, 1 - \{x_n\}\big),$$

so $d_0 = 3$, $d_1 = 2$, $d_2 = 4$, and δ_n is the distance from x_n to the nearest integer. The certificate at guard depth g uses $K = n + g + 1$ terms, an integer sum $S_{n,g} \in \mathbb{N}$, an error allowance $E_{n,g} = 4K + 2$, and returns $\lfloor (S_{n,g} \mathbin{\dot{-}} E_{n,g})/16^g \rfloor \bmod 16$ (Section 4).

Contributions.

(1) An independent Lean proof of (1) (Theorem 3.1). Earlier formal proofs exist in Coq and in Lean; see Section 2.

Date: 1 November 2026.

2020 *Mathematics Subject Classification.* 11Y60, 11J82, 68V20.

Key words and phrases. π, Bailey–Borwein–Plouffe formula, digit extraction, irrationality measure, Lean 4, Mathlib, formal verification.

(2) An integer-only digit certificate with a soundness theorem (Theorem 4.2) and a distance criterion: if $2E_{n,g} < 16^g\delta_n$ the bracket collapses (Lemma 4.3).
(3) A formal irrationality-measure bound at the denominators 16^k with exponent 570 and an explicit constant (Theorem 5.1).
(4) The main theorem, with two co-equal guards: for every n the certificate is proved to return d_n at every depth $g \geq 569n + \lfloor \log_{16}(n+1) \rfloor + 1 + B$, $B = 1\,795\,710$ (sharp form), and at every depth $g \geq 570n + 1\,795\,711$ (simplified form) (Theorem 6.1).
(5) Exact relations between the 0/F run after position n, the distance δ_n and the least sufficient guard depth, and the kernel-checked finite statement $\rho(77) = 0$, $\gamma(78) = 49$ (Section 7).

We make no claim about normality or about the statistics of the digits of π. Statements of the form "for every n" are theorems; statements about finite ranges of digits are labeled either as kernel-checked theorems or as computed numerics, and the latter are not used in any proof.

Status of the capped certificate. The development also contains an earlier capped certificate, which searches only a fixed set of guard depths; its checked ranges, the unchecked range and an explicit window where it is proved incorrect are described in Appendix A. It is not part of the main theorem.

2. Related work

Formal proofs about the BBP formula. Bertot, Rideau and Théry [2] formalized in Coq a proof of the BBP formula and a digit-extraction routine, and used it to compute the billionth hexadecimal digit of π inside Coq; the code is available as [11]. Their correctness statement for the digit routine is a partial-correctness statement: when the routine returns a digit, that digit is correct. As the authors note, a routine that never returns a digit would also satisfy it. They also note that the HOL Light library contains a formalization of the BBP formula. Our Theorem 4.2 plays the same role as their partial-correctness statement; the difference is Theorem 6.1, which supplies, for every n, a guard depth at which the check is proved to pass.

In Lean, Braams's lean-DLMF project [3] contains the theorem `hasSum_bbp_pi`, the BBP formula in Mathlib's `HasSum` form, now in the file `DLMF/Constants/BBP.lean`. The theorem was first committed publicly on 16 October 2026, in the file `AAR/Constants/BBP.lean`, which was renamed to its present path on 21 October 2026. Our proof of Theorem 3.1 was developed independently and uses a different route (an integral representation, see Section 3). We do not claim priority for the identity.

Irrationality measures of π. Mahler [6] proved the first explicit bound, $|\pi - p/q| > q^{-42}$ for all integers p and $q \geq 2$. Mignotte [7] lowered the exponent to about 20, and the best published bound we know of in a refereed journal is $7.103205334137\ldots$ by Zeilberger and Zudilin [12]. Our formal exponent 570 is far from these values; it comes from a deliberately simple instance of Mahler's method in which every constant is made explicit and checked in Lean (Section 5). What the application needs is not a small exponent but an explicit constant valid for every $k \geq 1$.

Other formal work. Mathlib [10] proves that π is irrational, and Eberl's Isabelle formalization [5] proves that π is transcendental. The Prove2Me platform [8] hosts Lean formulations of irrationality-measure bounds for π; the goal statements we inspected assert the existence of a threshold for every $\varepsilon > 0$ without giving explicit constants, so we do not use them.

3. The BBP identity

Theorem 3.1 (`bbpSeries_eq_pi`). *The series (1) converges and its sum is π.*

Outline of the formal proof. Write $p(x) = 4 - 2x^3 - x^4 - x^5$. Since $\int_0^1 x^{8k+j-1}\,dx = 1/(8k+j)$, the k-th summand equals $\int_0^1 (x^8/16)^k\, p(x)\,dx$. Dominated convergence for the geometric series gives

$$\sum_{k\ge 0} \frac{1}{16^k}\Big(\cdots\Big) = \int_0^1 \frac{p(x)}{1 - x^8/16}\,dx.$$

The factorizations $16 - x^8 = -(x^2-2)(x^2+2)(x^2+2x+2)(x^2-2x+2)$ and $p(x) = -(x-1)(x^2+2)(x^2+2x+2)$ reduce the integrand to $16(x-1)/\big((x^2-2)(x^2-2x+2)\big)$, which has the antiderivative $F(x) = 2\log(2-x^2) - 2\log(x^2-2x+2) + 4\arctan(x-1)$ on $[0,1]$. Then $F(1) - F(0) = \pi$ by the fundamental theorem of calculus. □

```
theorem bbpSeries_eq_pi : bbpSeries = Real.pi
```

4. The digit certificate and its soundness

Definition 4.1 (certificate). For $k, N \in \mathbb{N}$ let

$$t_k(N) = \left\lfloor \frac{4\cdot 16^N}{16^k(8k+1)} \right\rfloor \dot{-} \left\lfloor \frac{2\cdot 16^N}{16^k(8k+4)} \right\rfloor \dot{-} \left\lfloor \frac{16^N}{16^k(8k+5)} \right\rfloor \dot{-} \left\lfloor \frac{16^N}{16^k(8k+6)} \right\rfloor,$$

where $\dot{-}$ is truncated subtraction on $\mathbb{N}$, and the subtractions in $t_k(N)$ are grouped from the left. For $n, g \in \mathbb{N}$ put $K = n + g + 1$ and

$$S_{n,g} = \sum_{k<K} t_k(n+g) \in \mathbb{N}, \qquad E_{n,g} = 4K + 2, \qquad \mathrm{lo} = \left\lfloor \frac{S_{n,g} \dot{-} E_{n,g}}{16^g} \right\rfloor, \qquad \mathrm{hi} = \left\lfloor \frac{S_{n,g} + E_{n,g}}{16^g} \right\rfloor.$$

The guard *agrees at depth g* if $\mathrm{lo} = \mathrm{hi}$. The certified floor is lo and the certified digit is $\mathrm{lo} \bmod 16$.

In Lean these are `bbpSpigotTerm`, `bbpGuardedSum`, `bbpGuardErr`, `bbpGuardedLo`, `bbpGuardedHi`, `BBPGuardAgreesAt`, `bbpCertFloorAt` and `bbpCertDigitAt`. Everything is computed in $\mathbb{N}$. The lemma `bbp_term_cast_bounds` shows that the truncated subtractions in $t_k(N)$ never truncate and that

$$16^N a_k - 1 < t_k(N) < 16^N a_k + 3,$$

where a_k is the k-th summand of (1). Summing over $k < K$ and bounding the tail of the series by less than 1 gives

$$\mathrm{lo} \le x_n < \mathrm{hi} + 1 \qquad (\texttt{bbpGuarded_bracket}).$$

Theorem 4.2 (soundness; `BBPGuardAgreesAt.floor_pi`, `BBPGuardAgreesAt.digit_pi`). *For all $n, g \in \mathbb{N}$, if the guard agrees at depth g, then $\lfloor x_n \rfloor = \mathrm{lo}$ and $\mathrm{lo} \bmod 16 = d_n$.*

Proof. If $\mathrm{lo} = \mathrm{hi}$ the bracket gives $\mathrm{lo} \le x_n < \mathrm{lo} + 1$, so $\mathrm{lo} = \lfloor x_n \rfloor$; Theorem 3.1 replaces the series by π. □

Agreement is decidable, so for any particular (n, g) the hypothesis can be discharged by evaluation in the kernel. The next lemma gives a sufficient condition in terms of δ_n.

Lemma 4.3 (`guardAgreesAt_of_dist`). *Let $n, g \in \mathbb{N}$. If $2E_{n,g} < 16^g |x_n - m|$ for every $m \in \mathbb{Z}$, then the guard agrees at depth g. In particular this holds when $2E_{n,g} < 16^g \delta_n$.*

The factor 2 accounts for one $E_{n,g}$ from the error of $S_{n,g}$ and one from the width of the bracket. Since $2E_{n,g} = 8n+8g+12$ grows linearly in g while 16^g grows exponentially, Lemma 4.3 reduces the choice of a guard to a lower bound for δ_n.

5. An irrationality measure at the denominators 16^k

Theorem 5.1 (`BBPPi.MahlerSharp7b.pi_pow16_irrationality_measure`)**.** *For every integer $k \geq 1$ and every $p \in \mathbb{Z}$,*

$$|16^k\pi - p| \cdot 16^{570k} > 2^{-7182802} \cdot 16^k.$$

```
theorem BBPPi.MahlerSharp7b.pi_pow16_irrationality_measure :
    ∀ k : ℕ, 1 ≤ k → ∀ p : ℤ,
      |(16 : ℝ) ^ k * Real.pi - p| * (16 : ℝ) ^ (k * 570)
        > (1 / 2 ^ (7182801 + 1)) * (16 : ℝ) ^ k
```

Equivalently, with $q = 16^k$, $|\pi - p/q| > 2^{-7182802} q^{-570}$; so the statement is an irrationality-measure bound with exponent 570 and an explicit constant, restricted to the denominators that the digit application uses.

Structure of the formal proof. The proof follows Mahler's method [6], in the form used by Mignotte [7], with every constant explicit. We list the ingredients in the order in which they are combined.

(1) *Chebyshev bound.* From Mathlib's bound for the Chebyshev function ψ and the estimate $2\sqrt{x}\log x \leq 0.02x + 8760$ for $x \geq 1$ we obtain $\psi(x) \leq (\log 4 + 0.02)x + 8760$ (`LcmChebyshevSharp7b.psi_le_lin7`), hence $\operatorname{lcm}(1,\dots,n) \leq \exp(1.4063\,n + 8760)$ (`LcmChebyshevSharp7b.lcm_le_exp_lin7`). Here 1.4063 is a rational upper bound for $\log 4 + 0.02$.

(2) *Remainder.* For $n \geq 1000$ the auxiliary remainder terms satisfy a tail bound of the form $(8.55/(m+1))^J$ (`MahlerSharp7b.remainder_sharp7`); this uses $\pi e < 8.5398$. Specialized to $m = 141$ and $n = 4k + 12600$, it gives the smallness hypothesis of the next step for every $h \leq 141$ (`MahlerSharp7b.remainder_spec7`).

(3) *Mahler–Mignotte step.* A nonvanishing determinant argument (`MahlerSharp.pi_lower_bound_sharp`) shows that whenever $|\pi - p/q| \leq 1$,

$$1 \leq (2q)^m \, |\pi - p/q| \, T,$$

where T collects the size of the auxiliary coefficients.

(4) *Size of T.* With $m = 141$ and $n = 4k + 12600$, the bound $T \leq 2^{7182660+1716k}$ holds for every k (`MahlerSharp7b.T_spec7`).

(5) *Exponent arithmetic.* With $q = 16^k$ we have $(2q)^{141} = 2^{141+564k}$, so $|\pi - p/q| \geq 2^{-(141+7182660)} \cdot 2^{-(564+1716)k}$. Since $141 + 7182660 = 7182801$ and $564 + 1716 = 2280 = 4 \cdot 570$, this is $2^{-7182801}\, 16^{-570k}$. Passing to the strict inequality costs one more factor of 2, which gives the constant $2^{-7182802}$. The case $|\pi - p/q| > 1$ is immediate.

The remaining parameters ($m = 141$, the offset 12600, the constants 0.02 and 8760) were chosen numerically to keep the exponent and the constant moderate; the choice affects only the size of the constants, never correctness. □

Remark 5.2. The same development contains an earlier bound with exponent 658 and constant $2^{-1720001}$ (`MahlerSharp.pi_pow16_measure6`), used in Remark 6.2.

6. The main theorem: an explicit guard for every position

Combining Lemma 4.3 with Theorem 5.1 gives a guard depth that is sufficient for every n. Define

$$g(n) = 569n + \lfloor \log_{16}(n+1) \rfloor + 1 + B, \qquad B = 1\,795\,710.$$

Theorem 6.1 (main theorem; `BBPPi.floor_digit_eq_guard570`, `BBPPi.bbpDigit_eq_of_guard_ge`)**.**

(1) Sharp form. *For every $n \in \mathbb{N}$ the guard agrees at depth*

$$g(n) = 569n + \lfloor \log_{16}(n+1) \rfloor + 1 + 1\,795\,710,$$

so $\lfloor 16^n \pi \rfloor = \mathrm{lo}(n, g(n))$ and $\mathrm{lo}(n, g(n)) \bmod 16 = d_n$; the same holds at every depth $g \geq g(n)$ (`BBPPi.guardAgreesAt_of_guard570_le`).

(2) Simplified form. *For all $n, g \in \mathbb{N}$ with $g \geq 570n + 1\,795\,711$, the certificate at depth g returns $\lfloor 16^n \pi \rfloor$ and d_n.*

Both parts are proved in Lean as separate theorems. Part (1) gives the smaller depth; part (2) has a simpler formula. Since $\lfloor \log_{16}(n+1) \rfloor + 1 \leq n + 1$, we have $g(n) \leq 570n + 1\,795\,711$.

```
theorem BBPPi.floor_digit_eq_guard570 (n : ℕ) :
    ⌊(16 : ℝ) ^ n * Real.pi⌋ =
        (bbpCertFloorAt n (569 * n + (Nat.log 16 (n + 1) + 1) + 1795710) : ℤ) ∧
      bbpCertDigitAt n (569 * n + (Nat.log 16 (n + 1) + 1) + 1795710)
        = bbpHexDigitReal n

theorem BBPPi.bbpDigit_eq_of_guard_ge (n g : ℕ) (hg : 570 * n + 1795711 ≤ g) :
    ⌊(16 : ℝ) ^ n * Real.pi⌋ = (bbpCertFloorAt n g : ℤ) ∧
      bbpCertDigitAt n g = bbpHexDigitReal n
```

Outline. Write $g = 569n + L + j$ with $L = \lfloor \log_{16}(n+1) \rfloor + 1$, so that $n + 1 \leq 16^L$. The generic lemma `guard_agrees_of_const` shows that $\mathrm{BBPGuardAgreesAt}(n, g)$ holds for every n as soon as $j \geq 1$ and $(8 \cdot 570 + 20)(j+1) \leq 2^{-7182802}\, 16^j$. For $n \geq 1$ it combines Theorem 5.1 (with $k = n$), which gives $\delta_n > 2^{-7182802}\, 16^{-569n}$, with Lemma 4.3; for $n = 0$ it uses a direct lower bound for the distance from π to the nearest integer. The constant is computed exactly in Lean: $\ell = \lfloor \log_{16} \lceil 4580 \cdot 2^{7182802} \rceil \rfloor + 1 = 1795704$ (`guardLogL_570b`), since $4580 \cdot 2^{7182802} = 18320 \cdot 16^{1795700}$, and $j = \ell + \lfloor \log_{16}(2\ell) \rfloor + 1 = 1795704 + 5 + 1 = 1\,795\,710$ (`guardLogConst_570b`). Part (2) follows from part (1), monotonicity in g and $\lfloor \log_{16}(n+1) \rfloor \leq n$. □

Remark 6.2 (the exponent-658 guard). The earlier bound gives the guard $657n + \lfloor \log_{16}(n+1) \rfloor + 1 + 430009$ (`BBPPi.floor_digit_eq_guard658`), also proved for every n. It exceeds $g(n)$ by $88n - 1365701$, so it is the smaller of the two for $n \leq 15519$ and the larger for $n \geq 15520$. Both theorems hold for every n, so one can take the minimum.

How far the proved guard is from what is needed. Let $\nu(n)$ be the least g with $2E_{n,g} < 16^g \delta_n$ (in Lean, `piNeed`; it exists because π is irrational). Table 1 compares the proved guards with $\nu(n)$ at a few positions. The column $\nu(n)$ is *computed numerics*, obtained with `mpmath` at precision $4n + 400$ bits; it is not a Lean theorem and no result in this paper depends on it. The other columns are values of the formulas in Theorem 6.1 and Remark 6.2.

The proved guards are very far from $\nu(n)$. They are meant as a correctness guarantee for every position, not as a practical precision choice: in practice one runs the decidable check of Theorem 4.2 at a small depth, and Theorem 6.1 guarantees that a depth at which the check passes exists and is at most $g(n)$.

TABLE 1. Proved guard depths and the least sufficient depth $\nu(n)$. The last column is computed numerics, not a Lean theorem.

n	$g(n)$ (Thm. 6.1(1))	$570n + 1\,795\,711$ (Thm. 6.1(2))	exponent 658	$\nu(n)$ (numerics)
0	1 795 711	1 795 711	430 010	2
1	1 796 280	1 796 281	430 667	2
10	1 801 401	1 801 411	436 580	3
100	1 852 612	1 852 711	495 711	4
1000	2 364 713	2 365 711	1 087 012	4
10^4	7 485 714	7 495 711	7 000 013	5
10^5	58 695 715	58 795 711	66 130 014	6
10^6	570 795 715	571 795 711	657 430 014	7

7. Runs, distance and need

This section relates three quantities attached to a position n. All results are theorems for every n, except where an explicit range is stated.

Definition 7.1. Let $\rho(n)$ be the length of the maximal block of equal digits starting at position $n+1$ that consists only of 0s or only of Fs (`piRun`; $\rho(n) = 0$ if $d_{n+1} \notin \{0, \mathrm{F}\}$). For $s \in \mathbb{N}$, if some digit d_{s+k} with $k \geq 0$ equals 1, let $\gamma(s)$ be the least such k (`piGapSeq`).

The block in $\rho(n)$ is finite because π is irrational, so the Lean definition uses `Nat.find` together with Mathlib's `irrational_pi`. The Lean definition of $\gamma(s)$ returns 0 when no later 1 exists. Every statement below that involves γ carries the hypothesis that some later digit equals 1; we do not prove that this hypothesis holds for every s.

Theorem 7.2 (`piDist_run`)**.** *For every n, $16^{-(\rho(n)+1)} < \delta_n < 16^{-\rho(n)}$.*

Theorem 7.3 (`piNeed_bounds`)**.** *For every n, with $c_n = \lceil \log_{16}(8n + 8\nu(n) + 12) \rceil$,*

$$\rho(n) + c_n \leq \nu(n) \leq \rho(n) + c_n + 1.$$

In Lean c_n is `Nat.clog 16 (2 * bbpGuardErr n (piNeed n))`, and $2E_{n,\nu(n)} = 8n + 8\nu(n) + 12$. The inequalities in Theorem 7.2 are strict: $16^k \delta_n \neq 1$ for every k, because π is irrational.

Theorem 7.4 (`forced_chain`)**.** *For every n:*

1. *the digits $d_{n+1}, \ldots, d_{n+\rho(n)}$ are not equal to 1;*
2. *if some digit d_{n+1+k} $(k \geq 0)$ equals 1, then $\rho(n) \leq \gamma(n+1)$;*
3. *$16^{-(\rho(n)+1)} < \delta_n < 16^{-\rho(n)}$;*
4. *$\rho(n) + c_n \leq \nu(n) \leq \rho(n) + c_n + 1$;*
5. *the guard agrees at every depth $g \geq \nu(n)$.*

Corollary 7.5 (`long_run_forces`)**.** *If $L \leq \rho(n)$, then the digits $d_{n+1}, \ldots, d_{n+L}$ are not 1; if moreover some later digit equals 1, then $\gamma(n+1) \geq L$; and $\delta_n < 16^{-L}$ and $\nu(n) \geq L + c_n$.*

So a long run forces a long gap before the next 1, a small distance and a large need. The following explicit finite statement is kernel-checked.

Proposition 7.6 (`long_gap_short_run`)**.** *$\rho(77) = 0$ and $\gamma(78) = 49$.*

Proof. The lemma `cert_78_127` checks, by `decide +kernel`, that the certificate's guard agrees at positions $78, \ldots, 127$, that $d_{78} = 3$, that none of $d_{78}, \ldots, d_{126}$ equals 1, and that $d_{127} = 1$. Theorem 4.2 turns these checks into statements about the true digits. □

By Proposition 7.6, position 77 has a gap of 49 digits before the next 1 and no run of 0s or Fs, so, by Theorem 7.2, $\delta_{77} > 1/16$. Supporting lemmas in the same file include the countdown identities `piRun_succ`, `piRun_add` and `piGapSeq_countdown`, the bound `piRun_le_inter_one_gap`, and `piRun_pos_cases`, which records that a nonempty 0-run means x_n lies just above an integer and an F-run that it lies just below one.

8. The Lean development

Toolchain. The development uses Lean `leanprover/lean4:v4.29.0-rc8` and Mathlib at commit `816ff4582a01b10567d1d4ecb10be7c4ff92aab4`, as recorded in `lake-manifest.json`. The headline theorems of Sections 5 and 6 are in the namespace `BBPPi` (their full names are shown); all other proofs are in the namespace `FinitistProved` unless another namespace is shown. The earlier names of the renamed theorems (for example `bbp_digit_all_n_mu570b`) remain as deprecated aliases in `Thoughts/Deprecated.lean`. The results of Section 7 are in `FinitistProved.PiDigitStructure`, except the definition `piGapSeq`, which is in `Thoughts/Proved/BBPDigitDefs.lean`.

Axioms and decision procedures. For each theorem in Table 2, `#print axioms` reports no axioms beyond `propext`, `Classical.choice` and `Quot.sound` (`cert_78_127` uses only `propext`); in particular there is no `sorryAx`. The file `Thoughts/Axioms.lean` runs `#print axioms` on every theorem in the table that is in the default build, and the repository's continuous-integration job fails if any other axiom, any `sorry`, or any `native_decide` appears. The theorem of Appendix A for $n \le 10000$ is audited separately by `Thoughts/AxiomsHeavy.lean` after the heavy build. Finite computations are checked with `decide +kernel`, so they are evaluated by the kernel; `native_decide`, which would add the compiler to the trusted base, is not used.

Theorem names. Table 2 lists the Lean name and the file for each numbered result. All of them are on the `main` branch of the repository `metasecdev/math-proofs` [9] at the release tag `v1.0.1`, which contains only the BBP/π development.

Dependencies. The import closure of `PiDigitStructure.lean` contains seven modules of the development (the BBP algebra, series, tail, guarded-certificate and identity files, a definitions file, and the file itself). The import closures of the main theorem and of the capped-certificate results contain only modules of the BBP/π development (the certificate, the irrationality measure and the explicit Chebyshev bounds); the repository contains no unrelated modules.

Repository. The repository `metasecdev/math-proofs` pins Mathlib in the Lake configuration as well as in `lake-manifest.json`. Its continuous integration builds the default target, which contains every result in Table 2 except Theorem A.1, and runs the axiom check described above. The heavy certificates for Theorem A.1 need more memory than hosted runners provide and are built separately (`lake build BBPHeavyCerts`). The version cited here is the annotated tag `v1.0.1`, archived on Zenodo with DOI `10.5281/zenodo.23867088` (concept DOI for all versions: `10.5281/zenodo.23867087`) [9]. Its Lean sources are the same as at the earlier tag `v1.0.0`.

Acknowledgements

The author used AI assistants (Grok Bot) as tools for drafting Lean proofs, running computations and editing this text. All statements were checked by the Lean kernel or, where marked as numerics, by independent computation; the author is responsible for the content.

TABLE 2. Results and their Lean names in `metasecdev/math-proofs`, release tag `v1.0.1`. File paths are relative to `Thoughts/`. Theorem A.1 is in the separate Lake library `BBPHeavyCerts`, which GitHub CI does not build; it was checked by a local build on a MacBook Pro at commit `d8e418e`; the modules it depends on are unchanged at `v1.0.1`.

Result	Lean name	File
Thm. 3.1	`bbpSeries_eq_pi`	`Proved/BBPPiIdentity.lean`
Thm. 4.2	`BBPGuardAgreesAt.floor_pi`, `BBPGuardAgreesAt.digit_pi`	`Proved/BBPExplicitGuard.lean`
Lem. 4.3	`guardAgreesAt_of_dist`	`Research/BBPConditionalAllN.lean`
Thm. 5.1	`BBPPi.MahlerSharp7b.pi_pow16_irrationality_measure`	`Research/MahlerSharp7b.lean`
Thm. 6.1(1)	`BBPPi.guardAgreesAt_guard570`, `BBPPi.floor_digit_eq_guard570`, `BBPPi.guardAgreesAt_of_guard570_le`	`Research/BBPMu570b.lean`
Thm. 6.1(2)	`BBPPi.bbpDigit_eq_of_guard_ge`	`Research/BBPMu570b.lean`
Rem. 6.2	`BBPPi.floor_digit_eq_guard658`	`Research/BBPMu658.lean`
Thm. 7.2	`piDist_run`	`Research/PiDigitStructure.lean`
Thm. 7.3	`piNeed_bounds`	same file
Thm. 7.4	`forced_chain`, `long_run_forces`	same file
Prop. 7.6	`long_gap_short_run`, `cert_78_127`	same file
Thm. A.1	`BBPSpigotDigitCorrect_of_le_10000_unconditional`	`Proved/BBPDigitsUnconditionalExt10000.lean` (local build, `d8e418e`)
Thm. A.2	`not_BBPSpigotDigitCorrect_window'`, `not_BBPSpigotDigitCorrect_window`	`Research/BBPCapBracket.lean`, `Research/BBPCapRefutation.lean`

APPENDIX A. FUTURE WORK: THE CAPPED CERTIFICATE

A.1. **Definition.** The development's original certificate does not use an explicit guard. It tries the depths $4, 8, 12, \ldots, 64$ in order and stops at the first depth at which the guard agrees; if none agrees it uses depth 68 (`bbpCertGuard`, `bbpGuardSearch`). Its digit at position n is then compared with d_n; the statement that they are equal is `BBPSpigotDigitCorrect n`. When the search stops at an agreeing depth, Theorem 4.2 applies and the digit is correct. The question is what happens at the positions where no depth up to 64 agrees.

A.2. **What is proved.**

Theorem A.1 (checked by a local build)**.** *For every $n \le 10000$ the capped certificate returns d_n.*

In Lean this is `BBPSpigotDigitCorrect_of_le_10000_unconditional`, in the library `BBPHeavyCerts`. The proof uses a fast natural-number checker (`BBPFastGuard.lean`), proved to agree with the guard test, and checks in the kernel, in blocks, that the guard agrees for each $n \le 10000$ at the depth the search selects; it then applies Theorem 4.2. The bound 10000 reflects the cost of the check, not a mathematical limit.

This kernel check needs more than 10 GB of memory per Lean process, so it is not part of the GitHub continuous-integration build. It was checked by a local build of `BBPHeavyCerts` on a MacBook Pro (Intel Core i9-9980HK, 64 GB) at commit `d8e418e` on 31 October 2026: the build had no errors and no `sorry`, and the axiom audit `Thoughts/AxiomsHeavy.lean` reports only `propext`, `Classical.choice` and `Quot.sound` (report `reports/heavy-certs-2026-10-31.md`). The same argument for $n \le 2000$ (`BBPSpigotDigitCorrect_of_le_2000_unconditional`) is checked in the default build, which continuous integration runs.

Theorem A.2. *For every n with $2^{272} - 71 \le n$ and $5n + 348 \le 14 \cdot 2^{272}$, the capped certificate does not return d_n. In particular this holds for every $n \in [2^{272}, 2^{273}]$.*

In Lean these are `not_BBPSpigotDigitCorrect_window'` (general window) and `not_BBPSpigotDigitCorrect_window` (the interval $[2^{272}, 2^{273}]$).

The positions $10001 \le n \le 2^{272} - 72$ are unchecked: no Lean statement about the capped certificate at these positions is proved.

A.3. **Mechanism.** For $n \ge 1$ the lower end of the bracket satisfies, at every depth g,

$$16^g x_n - (5K + 3) - 16^g < \mathrm{lo} \cdot 16^g \le 16^g x_n - (K + 2) \qquad (\texttt{guardedLo_position}),$$

with $K = n+g+1$. Thus lo is biased downward from x_n by at least $(K+2)/16^g$. For $n \ge 2^{253}$ no depth up to 64 can agree (`bbpCertGuard_eq_68`), so the search uses $g = 68$. Once $16^{68} \le K + 2$, that is, $n \ge 2^{272} - 71$, the downward bias is at least one unit, while the upper condition $5K + 3 \le 14 \cdot 16^{68}$ keeps lo within the next lower integer; together they force $\mathrm{lo} \bmod 16 \ne d_n$ (`lo_digit_ne_of_window`). Below $2^{272} - 71$ the bias is smaller than one unit, and whether the capped digit equals d_n depends on the actual digits of π after position n (heuristically, not proved: on a run of about $68 - \log_{16} n$ zeros or Fs). These positions are unchecked.

A.4. **Directions.**

(1) *Sharper constants.* The exponent 570 and the constant B come from the factor 1.4063 in the bound for $\mathrm{lcm}(1, \dots, n)$ and from the other explicit choices in Section 5. Sharper explicit Chebyshev bounds would lower both.

(2) *Kernel-checked tables of $\nu(n)$.* The values in the last column of Table 1 could be turned into kernel-checked statements for explicit ranges, using Theorem 7.3 and the decidable check.

(3) *The heavy certificates.* Run the heavy build for Theorem A.1 in automated checking, and extend the kernel-checked range of the capped certificate beyond $n = 10000$.

References

1. David H. Bailey, Peter B. Borwein, and Simon Plouffe, *On the rapid computation of various polylogarithmic constants*, Math. Comp. **66** (1997), no. 218, 903–913.
2. Yves Bertot, Laurence Rideau, and Laurent Théry, *Distant decimals of π: formal proofs of some algorithms computing them and guarantees of exact computation*, J. Automat. Reason. **61** (2018), no. 1–4, 33–71, Preprint arXiv:1709.01743.
3. Bastiaan J. Braams, *lean-DLMF: file* `DLMF/Constants/BBP.lean`, https://github.com/bjbraams/lean-DLMF, Theorem `hasSum_bbp_pi`; first committed 16 October 2026 as `AAR/Constants/BBP.lean` (commit `4f09581`), renamed 21 October 2026 (commit `a5e2e4d`), accessed 1 November 2026.
4. Leonardo de Moura and Sebastian Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction – CADE 28 (Cham) (André Platzer and Geoff Sutcliffe, eds.), Lecture Notes in Computer Science, vol. 12699, Springer, 2021, pp. 625–635.
5. Manuel Eberl, *The transcendence of π*, Archive of Formal Proofs (2018), https://isa-afp.org/entries/Pi_Transcendental.html.
6. Kurt Mahler, *On the approximation of π*, Nederl. Akad. Wetensch. Proc. Ser. A **56** (1953), 30–42, = Indag. Math. **15**.
7. Maurice Mignotte, *Approximations rationnelles de π et quelques autres nombres*, Journées arithmétiques (Grenoble, 1973), Bull. Soc. Math. France, Mém., no. 37, Soc. Math. France, Paris, 1974, pp. 121–132.
8. Prove2Me, *Campaign: the irrationality measure of π*, https://prove2.me/campaigns/pi-irrationality-measure, Accessed 31 October 2026.
9. Brian Bernard Taylor, *A formally verified hexadecimal digit extraction for π with an explicit guard for every position (Lean 4 development)*, Zenodo, doi:10.5281/zenodo.23867088, 2026, Version 1.0.1, software. Source repository https://github.com/metasecdev/math-proofs, tag `v1.0.1`; concept DOI (all versions) 10.5281/zenodo.23867087.

10. The mathlib Community, *The Lean mathematical library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020) (New York), ACM, 2020, pp. 367–381.
11. Laurent Théry, *Plouffe: computing π decimals using the Plouffe formula in Coq*, https://github.com/thery/Plouffe, Software repository, accessed 31 October 2026.
12. Doron Zeilberger and Wadim Zudilin, *The irrationality measure of π is at most 7.103205334137…*, Mosc. J. Comb. Number Theory **9** (2020), no. 4, 407–419.